

MECHATRONICS AND IOT

ASSIGNMENT REPORT – 7

TITLE:

Design and develop an IoT-Based HVAC Temperature and Humidity Monitoring System. The system continuously monitors environmental conditions to improve HVAC performance and energy management.

TEAM MEMBERS:

HARISANKARAN S

(24MER013)

MUTHUKUMAR M

(24MER034)

MONESHWARAN C

(24MER032)

PROJECT OVERVIEW

Heating, Ventilation and Air Conditioning (HVAC) systems are widely used in industries, commercial buildings, hospitals, offices, laboratories, and residential buildings. Continuous monitoring of HVAC parameters is important for maintaining indoor comfort, reducing energy consumption, and preventing equipment failures. The proposed IoT-based HVAC monitoring system uses an ESP32 microcontroller, DHT22 sensor, Wi-Fi, MQTT communication, and ThingSpeak Cloud to monitor environmental conditions remotely. The sensor data is transmitted to the cloud, stored, and displayed through graphs for real-time monitoring and historical analysis.

PROBLEM STATEMENT

- ☐ Excessive room temperature
- ☐ High or low humidity
- ☐ Poor indoor environmental conditions
- ☐ Continuous operation of HVAC equipment
- ☐ Increased energy consumption
- ☐ Lack of remote monitoring
- ☐ Delayed detection of abnormal conditions

PROPOSED SOLUTION

The proposed system uses an ESP32 controller and DHT22 sensor to continuously monitor room temperature and humidity. The ESP32 collects and processes the sensor readings according to predefined threshold values. The data is transmitted through Wi-Fi and MQTT to a cloud platform, where it can be stored and analyzed. When temperature or humidity exceeds the specified limits, the system identifies the abnormal condition and can generate a warning or activate an alarm.

WORKING PRINCIPLE

The DHT22 sensor continuously measures room temperature and humidity. The ESP32 receives the sensor readings and compares them with predefined HVAC threshold values.

The processed data is:

DHT22 → ESP32 → Wi-Fi → MQTT → ThingSpeak Cloud

The current values can be displayed locally, while the cloud platform stores the data for remote monitoring and historical analysis. If the measured values exceed the specified limits, the system identifies the HVAC condition as NORMAL, WARNING, or CRITICAL.

SYSTEM ARCHITECTURE

HVAC Sensors → ESP32 Microcontroller → Wi-Fi Network → MQTT Communication → ThingSpeak Cloud → Cloud Storage & Graphs → Remote Mobile/Web Dashboard

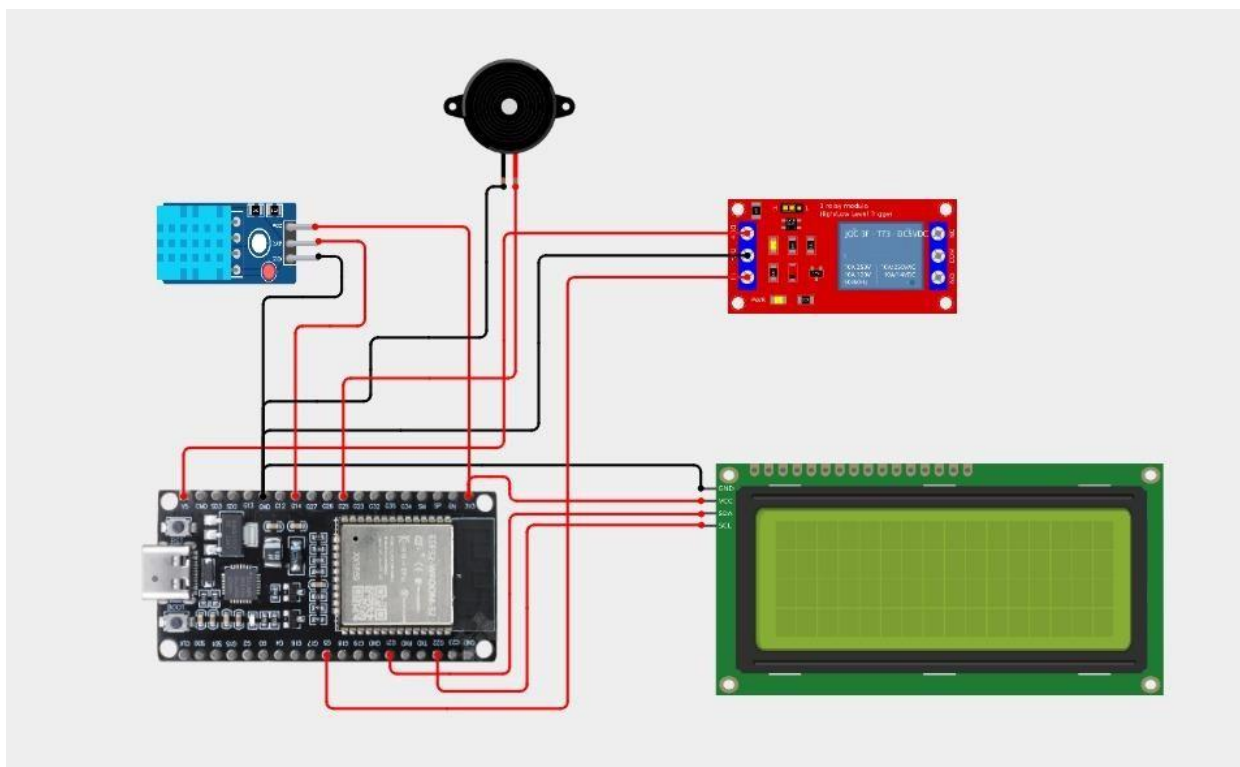
HVAC THRESHOLD VALUES

Parameter	Normal	Warning	Critical
Room Temperature	22–25 °C	25–28 °C	>28 °C
Humidity	40–60 %	60–70 %	>70 %

PARAMETERS MONITORED

Parameter	Sensor/Device	Purpose
Room Temperature	DHT22	Monitor room temperature
Humidity	DHT22	Monitor indoor humidity
Supply-Air Temperature	DS18B20	Monitor supply air
Return-Air Temperature	DS18B20	Monitor return air
Air Quality	MQ-135	Monitor air-quality conditions
Fan Status	Relay	Monitor fan operation
Compressor Status	Relay	Monitor compressor operation

CIRCUIT DESIGN



ALGORITHM

- **START**
- Initialize the ESP32 microcontroller.
- Initialize the DHT22 sensor.
- Initialize the LCD/display and buzzer.
- Connect the ESP32 to the Wi-Fi network.
- Connect to the MQTT broker.
- Read room temperature and humidity.
- Compare the readings with predefined HVAC threshold values.
- Determine the HVAC condition as **NORMAL, WARNING, or CRITICAL**.
- Display the readings on the display.
- Publish the sensor data using MQTT.
- Upload/store the data on ThingSpeak Cloud.
- Activate the buzzer when a critical condition is detected.
- Repeat the process continuously.
- **STOP**

CODING

```
#include "AdafruitIO_WiFi.h"

#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <DHT.h>

// =====
// WIFI DETAILS
// =====

#define WIFI_SSID    "OnePlus Nord CE5 7C5C"
#define WIFI_PASSWORD "qkts5737"

// =====
// ADAFRUIT IO DETAILS
// =====

#define IO_USERNAME   "Hair_13"
#define IO_KEY        "aio_EMRC40vci7DQ5Amz5kvOKh6WKHI0"

// Create Adafruit IO object
AdafruitIO_WiFi io(
  IO_USERNAME,
  IO_KEY,
```

```
WIFI_SSID,
WIFI_PASSWORD
);

// =====
// ADAFRUIT IO FEEDS
// =====

AdafruitIO_Feed *temperatureFeed =
  io.feed("temperature");

AdafruitIO_Feed *humidityFeed =
  io.feed("humidity");

AdafruitIO_Feed *hvacFeed =
  io.feed("hvac-status");

AdafruitIO_Feed *alertFeed =
  io.feed("hvac-alert");

// =====
// DHT11
// =====

#define DHT_PIN 4
#define DHT_TYPE DHT11

DHT dht(
  DHT_PIN,
  DHT_TYPE
);

// =====
// RELAY
// =====

#define RELAY_PIN 26

// Most relay modules are ACTIVE LOW
#define RELAY_ON LOW
#define RELAY_OFF HIGH

// =====
// BUZZER
// =====
```

```
#define BUZZER_PIN 25

// =====
// OLED
// =====

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64

Adafruit_SSD1306 display(
  SCREEN_WIDTH,
  SCREEN_HEIGHT,
  &Wire,
  -1
);

// =====
// VARIABLES
// =====

float temperature = 0;
float humidity = 0;

bool hvacStatus = false;
bool hvacAlert = false;

// =====
// HVAC SETTINGS
// =====

// HVAC turns ON at 28°C
float temperatureON = 28.0;

// HVAC turns OFF at 25°C
float temperatureOFF = 25.0;

// Humidity alert limit
float humidityLimit = 75.0;

// =====
// TIMING
// =====

unsigned long lastSend = 0;

const unsigned long sendInterval = 15000;
```

```

// =====
// SETUP
// =====

void setup()
{
  Serial.begin(9600);

  delay(1000);

  // -----
  // DHT11
  // -----

  dht.begin();

  // -----
  // RELAY
  // -----

  pinMode(RELAY_PIN, OUTPUT);

  digitalWrite(
    RELAY_PIN,
    RELAY_OFF
  );

  // -----
  // BUZZER
  // -----

  pinMode(BUZZER_PIN, OUTPUT);

  digitalWrite(
    BUZZER_PIN,
    LOW
  );

  // -----
  // OLED
  // -----

  Wire.begin(21, 22);

  if (!display.begin(

```

```

        SSD1306_SWITCHCAPVCC,
        0x3C))
    {
        Serial.println("OLED not detected!");
    }
    else
    {
        display.clearDisplay();

        display.setTextColor(
            SSD1306_WHITE
        );

        display.setTextSize(1);

        display.setCursor(10, 10);

        display.println(
            "HVAC MONITOR"
        );

        display.setCursor(10, 30);

        display.println(
            "Starting..."
        );

        display.display();
    }

    // -----
    // ADAFRUIT IO CONNECTION
    // -----

    Serial.println();
    Serial.println(
        "Connecting to Adafruit IO..."
    );

    io.connect();

    while (
        io.status() < AIO_CONNECTED
    )
    {
        Serial.print(".");

```



```

    delay(500);
}

Serial.println();
Serial.println(
    "Connected to Adafruit IO!"
);

display.clearDisplay();

display.setTextSize(1);

display.setCursor(0, 10);

display.println(
    "Adafruit IO"
);

display.setCursor(0, 30);

display.println(
    "Connected!"
);

display.display();

delay(2000);
}

// =====
// LOOP
// =====

void loop()
{
    // Keep Adafruit IO connection alive
    io.run();

    // -----
    // SEND DATA EVERY 15 SECONDS
    // -----

    if (
        millis() - lastSend >= sendInterval
    )
    {

```

```

lastSend = millis();

// =====
// READ DHT11
// =====

temperature =
  dht.readTemperature();

humidity =
  dht.readHumidity();

// -----
// CHECK SENSOR
// -----

if (
  isnan(temperature) ||
  isnan(humidity)
)
{
  Serial.println(
    "ERROR: DHT11 reading failed!"
  );

  return;
}

// =====
// SERIAL SENSOR OUTPUT
// =====

Serial.println();
Serial.println(
  "===== SENSOR DATA ====="
);

Serial.print(
  "Temperature: "
);

Serial.print(
  temperature
);

Serial.println(

```

```

    " C"
);

Serial.print(
    "Humidity: "
);

Serial.print(
    humidity
);

Serial.println(
    " %"
);

// =====
// HVAC CONTROL
// =====

if (
    temperature >= temperatureON
)
{
    hvacStatus = true;

    digitalWrite(
        RELAY_PIN,
        RELAY_ON
    );
}

else if (
    temperature <= temperatureOFF
)
{
    hvacStatus = false;

    digitalWrite(
        RELAY_PIN,
        RELAY_OFF
    );
}

// =====
// HVAC ALERT
// =====

```

```
if (
  temperature >= temperatureON ||
  humidity >= humidityLimit
)
{
  hvacAlert = true;

  digitalWrite(
    BUZZER_PIN,
    HIGH
  );
}

else
{
  hvacAlert = false;

  digitalWrite(
    BUZZER_PIN,
    LOW
  );
}

// =====
// SERIAL HVAC STATUS
// =====

Serial.println();
Serial.println(
  "===== HVAC STATUS ====="
);

Serial.print(
  "HVAC Status: "
);

if (hvacStatus)
{
  Serial.println("ON");
}
else
{
  Serial.println("OFF");
}
```

```

Serial.print(
  "HVAC Status Value Sent: "
);

Serial.println(
  hvacStatus ? 1 : 0
);

// =====
// SERIAL ALERT STATUS
// =====

Serial.println();
Serial.println(
  "===== HVAC ALERT ====="
);

Serial.print(
  "HVAC Alert: "
);

if (hvacAlert)
{
  Serial.println("ALERT");
}
else
{
  Serial.println("NORMAL");
}

Serial.print(
  "HVAC Alert Value Sent: "
);

Serial.println(
  hvacAlert ? 1 : 0
);

// =====
// SEND TEMPERATURE
// =====

Serial.println();
Serial.println(
  "===== ADAFRUIT IO ====="
);

```

```
Serial.print(  
  "Sending Temperature: "  
);
```

```
Serial.println(  
  temperature  
);
```

```
temperatureFeed->save(  
  temperature  
);
```

```
// =====  
// SEND HUMIDITY  
// =====
```

```
Serial.print(  
  "Sending Humidity: "  
);
```

```
Serial.println(  
  humidity  
);
```

```
humidityFeed->save(  
  humidity  
);
```

```
// =====  
// SEND HVAC STATUS  
// =====
```

```
int hvacValue =  
  hvacStatus ? 1 : 0;
```

```
Serial.print(  
  "Sending HVAC Status: "  
);
```

```
Serial.println(  
  hvacValue  
);
```

```
hvacFeed->save(  
  hvacValue
```

```

);

// =====
// SEND HVAC ALERT
// =====

int alertValue =
  hvacAlert ? 1 : 0;

Serial.print(
  "Sending HVAC Alert: "
);

Serial.println(
  alertValue
);

alertFeed->save(
  alertValue
);

// Give Adafruit IO time to process queued messages
io.run();

Serial.println();
Serial.println(
  "All data sent to Adafruit IO."
);

Serial.println(
  "=====
);

// =====
// OLED DISPLAY
// =====

display.clearDisplay();

display.setTextColor(
  SSD1306_WHITE
);

display.setTextSize(1);

// Temperature

```

```
display.setCursor(0, 0);

display.print("T:");

display.print(
  temperature,
  1
);

display.print("C ");

// Humidity
display.print("H:");

display.print(
  humidity,
  0
);

display.println("%");

// HVAC Status
display.setCursor(0, 20);

display.print("HVAC: ");

if (hvacStatus)
{
  display.println("ON");
}
else
{
  display.println("OFF");
}

// Alert
display.setCursor(0, 35);

display.print("Alert: ");

if (hvacAlert)
{
  display.println("ALERT!");
}
else
{
```



```

        display.println("Normal");
    }

    // IoT
    display.setCursor(0, 50);

    if (
        io.status() >= AIO_CONNECTED
    )
    {
        display.println(
            "IoT: Connected"
        );
    }
    else
    {
        display.println(
            "IoT: Offline"
        );
    }

    display.display();

    Serial.println(
        "====="
    );
}
}

```

SYSTEM WORKING

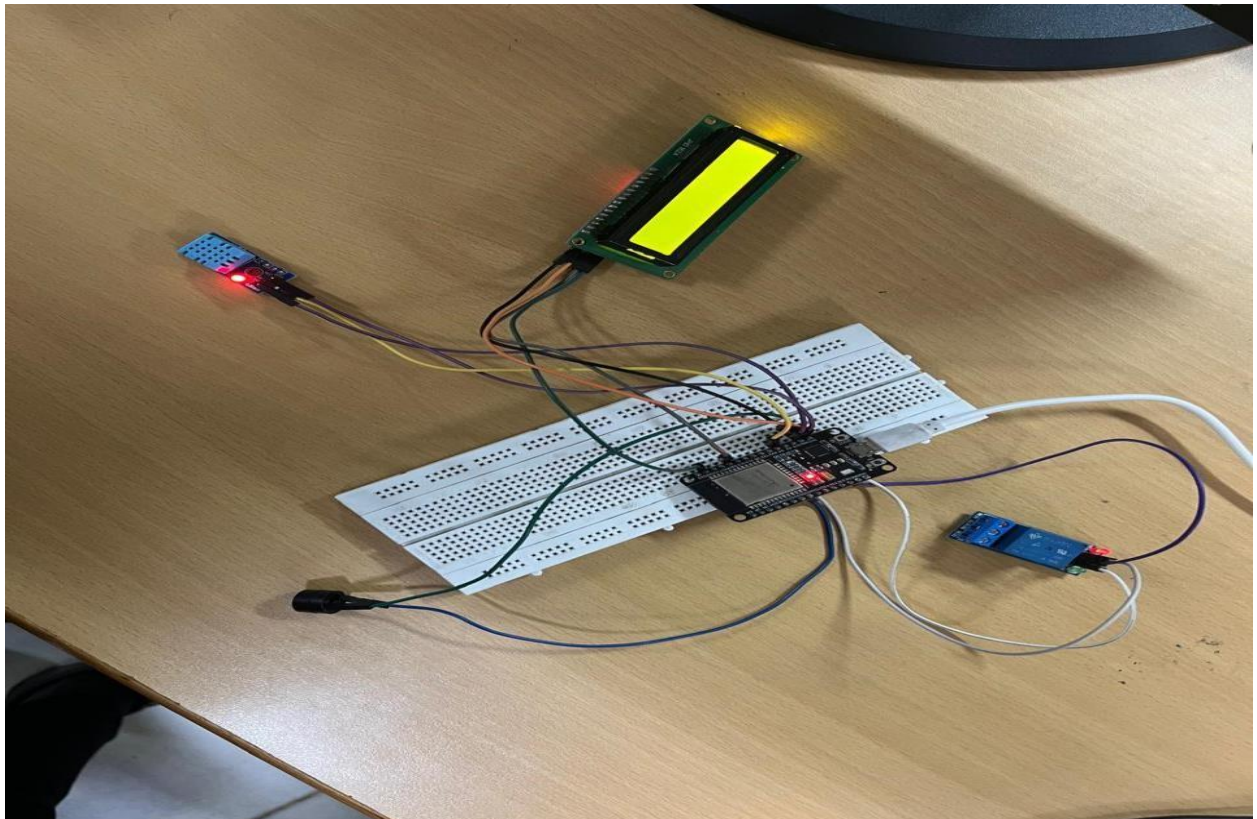
The system uses an ESP32 microcontroller to continuously monitor environmental conditions. The DHT22 measures room temperature and humidity. The ESP32 collects the sensor readings and compares them with predefined threshold values. The current readings are displayed locally. The ESP32 then sends the sensor data through Wi-Fi using MQTT communication to the cloud platform. ThingSpeak stores the data and provides graphs for real-time monitoring and historical analysis. If the temperature or humidity exceeds the specified limits, the system identifies the condition as WARNING or CRITICAL and activates the buzzer during critical conditions. The monitoring process continues continuously.

COMPONENTS

S.No.	Component	Quantity	Purpose
1	ESP32 Dev Board	1	Main controller
2	DHT22 Sensor	1	Room temperature & humidity
3	DS18B20 Sensor	2	Supply and return air temperature
4	MQ-135 Air Quality Sensor	1	Air-quality monitoring
5	Relay Module	2	Fan and compressor control/status
6	16×2 I2C LCD	1	Display HVAC parameters
7	Buzzer & LEDs	1 set	Warning indication
8	Breadboard & Jumper Wires	1 set	Circuit connections
9	Power Supply	1	Power source

PROTOTYPE IMPLEMENTATION

The prototype is designed around an ESP32 controller and environmental sensors. The DHT22 is connected to the ESP32 for collecting room temperature and humidity data. The display provides real-time visualization of HVAC parameters, while the buzzer provides an alert during critical conditions. The ESP32 communicates through Wi-Fi and MQTT. The monitored HVAC data is transmitted to the cloud platform for storage and visualization. Therefore, the prototype provides both local monitoring and remote cloud-based monitoring.



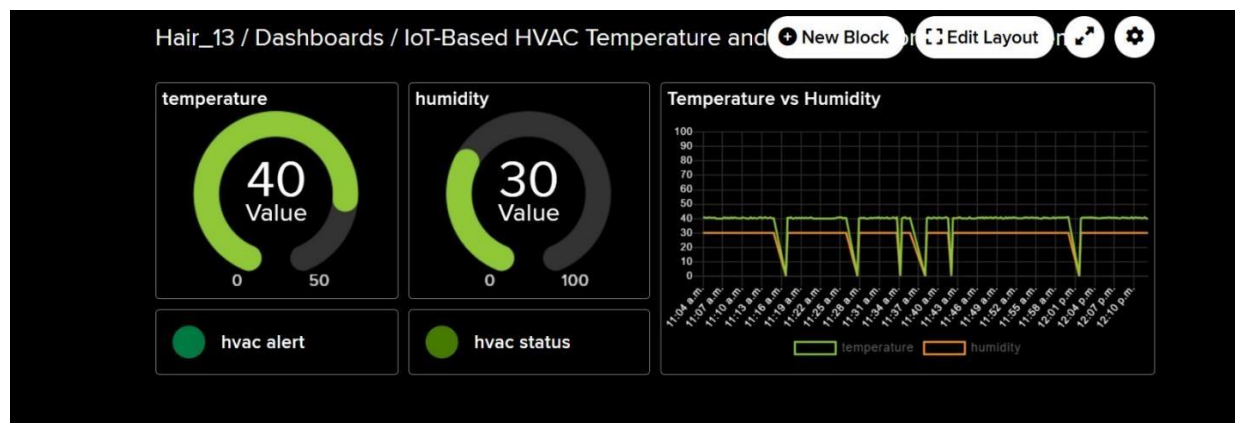
RESULTS & PERFORMANCE ANALYSIS

The developed system provides real-time monitoring of HVAC environmental parameters. The sensor readings are compared with predefined threshold values to identify normal, warning, and critical operating conditions.

Parameter	Normal	Warning	Critical
Room Temperature	22–25 °C	25–28 °C	>28 °C
Humidity	40–60 %	60–70 %	>70 %
Supply-Air Temperature	14–18 °C	18–20 °C	>20 °C
Return-Air Temperature	20–25 °C	25–28 °C	>28 °C
Air Quality	<100	100–150	>150

PERFORMANCE

- Provides real-time HVAC temperature and humidity monitoring.
- Uses MQTT-based wireless communication.
- Provides remote monitoring through ThingSpeak Cloud.
- Automatically detects abnormal temperature and humidity conditions.
- Provides a local buzzer alert during critical conditions.
- Stores historical sensor data for HVAC performance analysis.
- Provides graphical visualization of monitored data.
- Helps identify abnormal HVAC operating conditions quickly.
- Provides a low-cost prototype suitable for educational and small-scale IoT applications.



iotexp-2.ino

595

596

597

598

599

600

601

602

603

604

605

"IoT: Offline"

);

}

display.display();

Serial.println(
"====="

);

}

}

Output

Serial Monitor

X

Message (Enter to send message to 'ESP32 Dev Module' on 'COM8')

===== HVAC ALERT =====

HVAC Alert: ALERT

HVAC Alert Value Sent: 1

===== ADAFRUIT IO =====

Sending Temperature: 40.90

Sending Humidity: 30.00

Sending HVAC Status: 1

Sending HVAC Alert: 1

All data sent to Adafruit IO.

=====

=====